

Learning reusable computation

Lecture 5 — Mathematical Foundations of Modern AI

How can one next-token predictor execute many different programs?

Neural controller with external memory

Scaffolded neural interpreter

Self-contained neural interpreter

Dale Schuurmans (2023) ■ Ruize Xu et al. (2026)

What could prediction teach a model to do?

Lectures 3–4: next-token log loss fits a distribution over sequences.

$$-\log q_{\theta}(x_{1:T}) = \sum_{t=1}^T -\log q_{\theta}(x_t \mid x_{<t}).$$

Suppose the sequences describe computations, with successive states

$$\sigma_0 \longmapsto \sigma_1 \longmapsto \cdots \longmapsto \sigma_L.$$

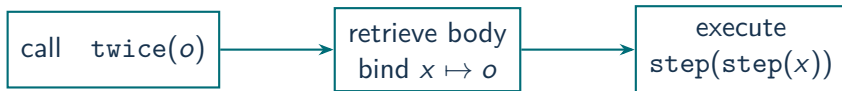
The same transition rule can appear in many different tasks.

Today: specify that rule, expose it in training data, and ask whether a transformer learns to apply it in new computations.

A name lets us reuse a definition

Once a procedure has a name, later instructions can refer to its body:

$$\Delta(\text{twice}) = (x, \text{step}(\text{step}(x))).$$



The model must connect a symbol to its definition, preserve bindings, and resume the caller after the procedure finishes.

Can predictive training recover these reusable operations?

The target is a shared interpreter

Fix a deterministic decoder. Let p be a program and x its input.

Definition 1 (Exact execution)

A system with fixed neural parameters θ executes a program family if

$$\text{Run}_{\theta}(p, x) \simeq p(x) \quad \text{for every allowed } (p, x).$$

Here $\simeq$ means the same returned value, or divergence on both sides.

The run may use many model calls and writable memory. The system is *universal* when it executes an effective encoding of every Turing machine.

Different programs; the same learned execution mechanism.

Universality and learned execution

We will compare three ways to divide the execution between a neural model and its surrounding system.

Our name	Execution mechanism
Neural controller with external memory	External code supplies operands and applies the selected commands.
Scaffolded neural interpreter	The model retrieves and updates language state. A fixed scaffold contracts calls.
Self-contained neural interpreter	One continuous generation executes the program, with intermediate tokens or directly.

Universality: can the whole system execute arbitrary computations?

Learning: how much of the execution mechanism can the model learn?

Schuurmans: 29 local cases suffice

A fixed universal Turing machine has 15 control states and 2 tape symbols. Schuurmans implements its controller through 29 distinct LLM prompts.



The paper prints Flan-U-PaLM 540B's greedy responses for all 29 prompts. The tape is stored externally.

Why can checking a finite table certify arbitrarily long computations?

Schuurmans (2023), Sections 2–5. A sign discrepancy between its table and printed prompt program is documented in backup.

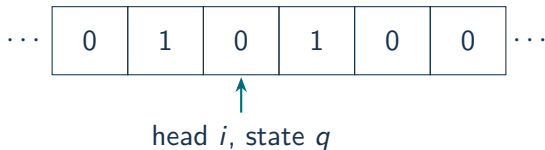
A Turing machine changes one tape cell at a time

Let $\mathcal{Q}$ be a finite set of control states and Σ a finite tape alphabet. An instruction is

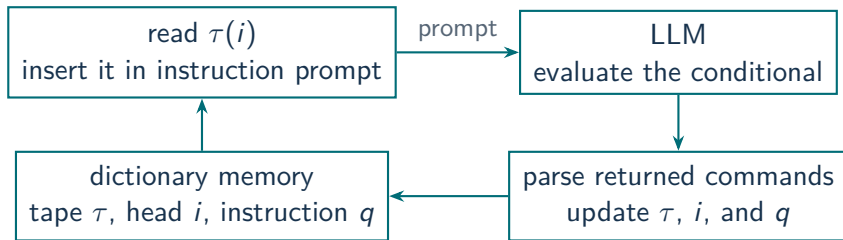
$$\delta(q, b) = (b', d, q'), \quad d \in \{-1, +1\}, \quad \text{or} \quad \delta(q, b) = \text{halt}.$$

A *configuration* is $\sigma = (q, i, \tau)$, where $i \in \mathbb{Z}$ is the head position and $\tau : \mathbb{Z} \rightarrow \Sigma$ is blank outside a finite set.

$$(q, i, \tau) \mapsto (q', i + d, \tau[i \mapsto b']), \quad (b', d, q') = \delta(q, \tau(i)).$$



The external code resolves the memory addresses



The head address and the rest of the tape stay outside the neural input.

Longer computations require more cycles and possibly more tape.

The neural controller continues to receive the same finite set of inputs.

Schuurmans (2023), Sections 2–5.

One prompt implements one conditional instruction

For control state A :

$$\delta(A, 0) = (0, +1, B), \quad \delta(A, 1) = (1, +1, A).$$

Prompt after reading the tape

```
result = (write 0, right, state B)
if 1 == 1:
    result = (write 1, right, state A)
return result
```

Parsed result

$$(b', d, q') = (1, +1, A).$$

The external code writes 1 at the current address and increments the address.

The model selects commands; the memory implementation applies them.

Simplified notation for Schuurmans's prompt program, Section 4. The original returns assignment strings with memory-substitution markers.

Correct local transitions give correct full executions

Let $N_\theta(q, b)$ be the parsed neural response, and define

$$\text{Step}_\theta(q, i, \tau) = \text{Update}((q, i, \tau), N_\theta(q, \tau(i))).$$

Proposition 1 (Exact simulation)

If $N_\theta(q, b) = \delta(q, b)$ for every controller input, then, from every initial tape, the neural system and the Turing machine have identical configurations after every step, including the same halting behavior.

Proof. They start in the same configuration. If their configurations agree at step t , they read the same bit and select the same instruction. The same update gives agreement at step $t + 1$. Induct on t .

For a universal machine, this transfers universality to the whole system.

A fixed finite test certifies the neural component

Let $\mathcal{U}$ contain the controller's distinct prompts, and let $N^*(u)$ be the command prescribed by the transition table.

$$|\mathcal{U}| = 29, \quad N_\theta(u) = N^*(u) \quad \text{for every } u \in \mathcal{U}.$$

Together with the correct read/write interface, these checks imply exact simulation for every initial tape and every execution length.

The neural context and verification set stay fixed as the tape grows.

A neural network can also learn this finite table from labeled examples. The learning problem is independent of the eventual memory and time requirements.

The finite-table learning bound is in backup. Schuurmans tests an already pretrained model. His printed sign discrepancy must be corrected and the revised prompt checked to certify the stated machine.

What if the model must find the operands itself?

In a programming language, an instruction may require retrieving

$$\underbrace{\Delta(f)}_{\text{procedure body}}, \quad \underbrace{\rho(x)}_{\text{current variable binding}}, \quad \underbrace{H(o, a)}_{\text{current attribute value}}.$$

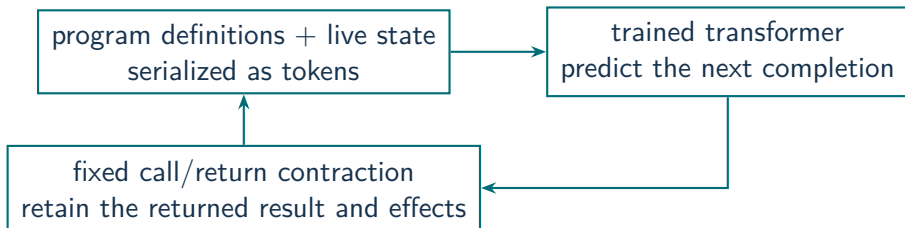
The relevant entry depends on the program, the active call, and previous writes.

MicroPy trains the transformer to perform these retrievals and updates.

The input includes the definitions and execution state. The output describes the next part of the execution.

A finite list of operations is now applied to a large family of structured inputs.

MicroPy: a scaffolded neural interpreter



The reported 59.5M-parameter model executes held-out programs for bit operations, arithmetic, and SAT tasks. Training uses next-token prediction on interpreter traces.

The model performs retrieval and rewriting. The scaffold manages call contraction.

Xu et al. (2026), Sections 2–3.

The execution state has four distinct components

A fixed procedure table maps names to parameters and bodies:

$$\Delta(f) = ((x_1, \dots, x_k), e).$$

For this program, an execution configuration is

$$\sigma = \langle H, \rho, e, K \rangle.$$

$H : \mathcal{O} \times \mathcal{A} \rightarrow \mathcal{O}$ heap: object attributes and their values

$\rho : \mathcal{V} \rightarrow \mathcal{O}$ local environment: variable bindings

$e \in \text{Expr}$ expression currently being evaluated

K continuation stack: what remains to do

Here $\mathcal{O}, \mathcal{A}, \mathcal{V}$ are object, attribute, and variable names. The maps and stack are finite.

Lecture formalization of MicroPy's semantic state. We inline expression definitions; the paper stores them under separate expression names.

MicroPy's primitive operations

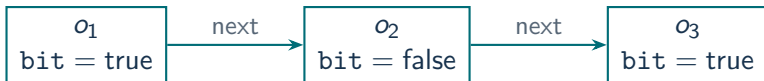
Expression	Effect of evaluation
$\text{IF}(e_1, e_2, e_3)$	Evaluate e_1 ; choose e_2 for true, otherwise e_3 .
$\text{EQUAL}(e_1, e_2)$	Test whether the resulting objects are identical.
$\text{HasAttr}(e, a)$	Test whether the resulting object has attribute a .
$\text{LookupAttr}(e, a)$	Read that attribute.
$\text{Assert}(e_1, a, e_2)$	Set the first object's attribute a to the second value.
$\text{SEQ}(e_1, e_2)$	Evaluate e_1 for its effects, then return the value of e_2 .
$\text{Try}(e_1, e_2)$	Try e_1 ; on fail, roll back its writes and evaluate e_2 .

Also: object constants, $\text{LookupVar}(x)$, and $\text{App}(f, e_1, \dots, e_k)$.

Xu et al., Section 2.3. In our formalization, arguments are evaluated left to right and a successful write returns unit. Rollback rules are in backup.

One procedure can flip a list of any finite length

The heap represents a bit string by linked objects. The final object has no next attribute.



```
def flip(x):  
    x.bit = IF(x.bit, false, true)  
    if HasAttr(x, next):  
        return flip(x.next)  
    else:  
        return unit
```

Readable surface notation for the lecture's MicroPy program; the exact primitive expression is in backup.

One iteration: retrieve, choose, write, recurse

Start with $\rho(x) = o_1$ and $H(o_1, \text{bit}) = \text{true}$.

$$\begin{aligned}\text{LookupVar}(x) &\mapsto o_1, \\ \text{LookupAttr}(o_1, \text{bit}) &\mapsto \text{true}, \\ \text{IF}(\text{true}, \text{false}, \text{true}) &\mapsto \text{false}, \\ \text{Assert}(o_1, \text{bit}, \text{false}) &\mapsto \text{unit}, \quad H' = H[(o_1, \text{bit}) \mapsto \text{false}].\end{aligned}$$

Since $H'(o_1, \text{next}) = o_2$, the next call is $\text{flip}(o_2)$.

A longer list repeats the same rule types with different bindings.

The next lookup must still find the correct object and its current value.

A procedure call creates a new environment

Suppose $\Delta(f) = ((x_1, \dots, x_k), e_f)$ and the arguments have evaluated to objects $o_1, \dots, o_k$. Write $\mapsto^*$ for zero or more steps.

$$\begin{aligned} &\langle H, \rho, \text{App}(f, o_1, \dots, o_k), K \rangle \\ &\quad \mapsto^* \langle H, [x_1 \mapsto o_1, \dots, x_k \mapsto o_k], e_f, \text{Return}(\rho) :: K \rangle. \end{aligned}$$

When the body returns o with heap H' :

$$\langle H', \rho_f, o, \text{Return}(\rho) :: K \rangle \mapsto \langle H', \rho, o, K \rangle.$$

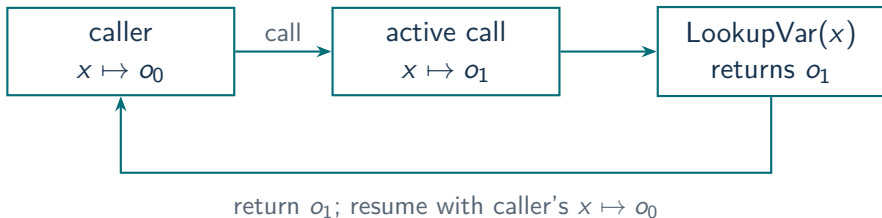
Restore the caller's bindings; preserve the procedure's heap updates.

The stack records the continuation. Recursion applies this same call rule again.

An ordinary environment/continuation semantics for the lecture core. MicroPy serializes these operations into several supervised rewrites.

The same variable name can have different bindings

For $\Delta(\text{id}) = (x, \text{LookupVar}(x))$, call $\text{id}(o_1)$ from an environment where x denotes o_0 .



Retrieval must respect scope, not just match the name x .

Attribute retrieval must likewise select the latest applicable write to (o, a) .

PENCIL deletes a completed subcomputation

Once a call has produced its returned payload γ , apply

$$\underbrace{\alpha}_{\text{retained context}} \quad [\text{call}] \quad \underbrace{\beta}_{\text{completed work}} \quad \rightarrow \quad \underbrace{\gamma}_{\text{result and effects}} \quad [\text{ret}] \quad \mapsto \quad \alpha \gamma.$$

The matching active call frame is removed. The payload includes the value and any heap changes needed by the continuation.



the external rule removes this frame

Keep the information needed to continue; reclaim the intermediate work.

PENCIL contraction as used by Xu et al., Equation (1).

Execution time and active memory can grow differently

Let L count execution steps, and let S be the largest live configuration size, including definitions, heap, and active calls.

Retain the complete transcript

Every step contributes tokens. Even a loop over a small state keeps increasing the context.

retained history grows with L .

Retain the live configuration

Completed calls release their frames. A tail call can replace the current frame instead of adding one.

required state is governed by S .

For a list traversal, the list must remain available; old work need not.

PENCIL provides the contraction mechanism. The returned state representation determines how much memory is actually reclaimed.

Fixed context bounds the interpreter's memory

Suppose the entire persistent state is a string of at most W tokens from a vocabulary of size V . Each next call depends only on this string. There are at most $N_W = \sum_{w=0}^W V^w$ persistent states.

Proposition 2 (Finite-state execution)

Under deterministic decoding, a run with more than N_W nonterminal states repeats a state and then cycles forever.

Proof. Pigeonhole principle followed by determinism.

At fixed context, this implementation is not a universal interpreter.

PENCIL makes room for more execution steps by removing completed work. Unbounded working memory still requires growing context and available objects.

A finite rule set can have an unbounded input domain

Schuurmans's controller always receives a prompt in the same 29-element set. MicroPy's lookup rule must handle contexts containing increasingly many bindings, scopes, positions, and intervening writes:

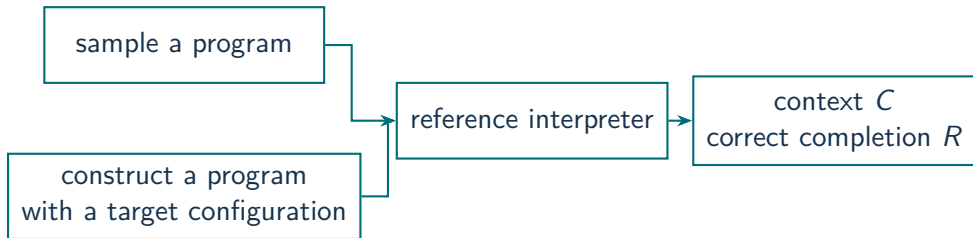
LookupVar(x) must retrieve the binding in the active call.

At context bound W , all token strings form a finite set of size N_W . Increasing the memory bound increases this input domain.

There is no fixed exhaustive input test as the allowed context grows.

A correctness proof would need an argument that applies to every context size. Learning can exploit the shared rules, but must generalize their retrieval and rewriting to new structured inputs.

Training data exposes the interpreter's intermediate work



Random programs provide diverse executions. Targeted program generation exposes rare cases: shadowed variables, intervening writes, and nested continuations.

Choosing intermediate targets and their coverage is part of the method.

Xu et al. (2026), Sections 2–3.

The objective is ordinary next-token log loss

For training pairs (C_i, R_i) , $i = 1, \dots, n$, minimize

$$\hat{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^{|R_i|} -\log q_{\theta}(R_{i,j} \mid C_i, R_{i,<j}).$$

C_i is the observed interpreter context; R_i is the reference completion. Teacher forcing supplies its preceding tokens during training.

At execution time, generated tokens determine the next completion and hence the next context after contraction.

A short completion may require searching a large, structured context.

The neural task includes locating the correct binding, not just choosing an operation from a short list.

The reported executions are substantially longer

Training program executions are capped at **128 trace lines**. The decoder-only transformer has **59.5 million parameters**.

Test task	Instances	Max. lines	Max. context	Token accuracy
Bit copying	9	422	725	100%
Bit flipping	9	572	774	100%
Binary addition	9	2,365	1,514	100%
Binary multiplication	9	7,552	2,197	100%
SAT solving	180	5,020	2,091	100%
SAT verification	36	628	1,398	100%

252 test instances; 224,708 stepwise prompt/completion pairs.

7,552/128 $\approx$ 59: familiar rules work in much longer runs.

Xu et al., Tables 1–2. Context is measured in tokens per step; reported accuracy is token-level. Test sizes and metric details are in backup.

MicroPy generalizes over execution-trace length

Quantity	What increases?	What must generalize?
Execution length L	Number of interpreter steps	Repeated application of the learned rules
Context length W	Definitions, objects, live bindings and calls	Retrieval and rewriting with more live information
Completion length r	Tokens generated for one supervised step	Construction of a longer local response

The reported 59-fold increase concerns L .

Test contexts reach 2,197 tokens. The paper gives no corresponding training context-length range establishing extrapolation in W .

Xu et al., Section 3 and Tables 1–2. Local completion-length statistics are not reported.

Exact local prediction has a strong consequence

Fix a deterministic decoder and the call/return protocol. Let

$$C_{t+1} = h(C_t, R_t),$$

where h appends completion R_t and performs the prescribed contractions.

Proposition 3 (Exact-prefix induction)

If the decoder predicts every token correctly on every reference prefix of the required completions, including their stopping markers, then it generates the entire reference execution exactly.

Proof. The first token agrees. Agreement through token $j - 1$ gives the reference prefix for token j . The completed response agrees, so h produces the next reference context. Repeat.

Literal zero errors is qualitatively stronger than a small average error.

Self-contained execution: one continuous generation

Supply the program p and input x . Ordinary autoregressive decoding produces the execution and answer, without interpreter-specific external operations.

With chain of thought (CoT):

$$\text{enc}(p, x) \mapsto \underbrace{r_1, \dots, r_\ell}_{\text{intermediate tokens}} \mapsto \text{enc}(p(x)).$$

The model must represent the evolving state and continue the computation using its generated tokens.

Direct output:

$$\text{enc}(p, x) \mapsto \text{enc}(p(x)).$$

The answer tokens carry the output, with no separate intermediate trace.

Self-contained describes execution. Training may still use execution traces.

Intermediate tokens provide additional computational time

Let d be transformer depth, a the number of answer tokens, and ℓ the number of intermediate tokens. At fixed input length, the sequential neural depth available through decoding scales as

direct output: $O(da)$,

with CoT: $O(d(\ell + a))$.

One continuous generation can therefore perform many successive computations. Retaining its full transcript also makes the context grow with ℓ .

A short answer can require a long computation.

Direct execution can exploit shortcuts for particular programs. A fixed-depth model with a fixed short response has no adjustable execution-time budget. CoT supplies one through the number of intermediate tokens.

Computational-resource connection: Merrill and Sabharwal (2024), [The Expressive Power of Transformers with Chain of Thought](#). Formal complexity results return in later lectures.

The execution interface determines the tradeoffs

Approach	What it provides	What it requires
Controller with external memory	Fixed neural context and a finite correctness test	External code handles all memory addressing and updates.
Scaffolded interpreter	Learned structured retrieval and reclaiming completed work	Live state fits in context. Retrieval must handle new configurations.
Self-contained, with CoT	Execution through ordinary generation, with adjustable time	The model manages state in its trace. Retained context grows with that trace.
Self-contained, direct output	Short responses when the model can compute the answer directly	The computation must fit the input-processing and output-generation budget.

Reducing external support transfers more of the execution to the learned model.

What these constructions teach us

Universality is a property of the whole system. A fixed neural controller can suffice when external memory grows. MicroPy's context must grow to support unbounded live state.

The learning problems differ. Schuurmans reduces neural correctness to a fixed finite table. MicroPy learns retrieval and rewriting over structured contexts, and reports generalization to longer execution traces.

The more ambitious target is self-contained execution. The model carries out the computation through ordinary generation. CoT provides intermediate steps. Direct output offers fewer steps when the answer is short.

Can we learn the execution rules and make them reliable as the required time and memory grow?

Reading

Dale Schuurmans (2023).

Memory Augmented Large Language Models are Computationally Universal.

[arXiv:2301.04589](#)

Sections 2–5: memory interface, universal machine, prompt program, finite test.

Ruize Xu, Chenxiao Yang, Yanhong Li, David McAllester (2026).

Training Transformers as a Universal Computer.

[arXiv:2604.25166v1](#)

Section 2: execution model. Section 3: data and results. Appendices B–D: concrete traces and definitions.

The following backup slides provide the lecture's full core semantics, proof details, and additional questions.

The universal machine used by Schuurmans

Each entry is (written bit, head movement, next state).

State	Read 0	Read 1
<i>A</i>	(0, +1, <i>B</i>)	(1, +1, <i>A</i>)
<i>B</i>	(1, +1, <i>C</i>)	(1, +1, <i>A</i>)
<i>C</i>	(0, -1, <i>G</i>)	(0, -1, <i>E</i>)
<i>D</i>	(0, -1, <i>F</i>)	(1, -1, <i>E</i>)
<i>E</i>	(1, +1, <i>A</i>)	(1, -1, <i>D</i>)
<i>F</i>	(1, -1, <i>D</i>)	(1, -1, <i>D</i>)
<i>G</i>	(0, +1, <i>H</i>)	(1, -1, <i>G</i>)
<i>H</i>	(1, -1, <i>I</i>)	(1, -1, <i>G</i>)

State	Read 0	Read 1
<i>I</i>	(0, +1, <i>A</i>)	(1, -1, <i>J</i>)
<i>J</i>	(1, -1, <i>K</i>)	halt
<i>K</i>	(0, +1, <i>L</i>)	(1, +1, <i>N</i>)
<i>L</i>	(0, +1, <i>M</i>)	(1, +1, <i>L</i>)
<i>M</i>	(0, -1, <i>B</i>)	(1, +1, <i>L</i>)
<i>N</i>	(0, -1, <i>C</i>)	(0, +1, <i>O</i>)
<i>O</i>	(0, +1, <i>N</i>)	(1, +1, <i>N</i>)

Blank 0; initial state *A*. The tape encodes the program and its input. State *F* uses one prompt for both bits: 29 prompts for 30 state/bit pairs.

Schuurmans, Table 1 and Section 4; machine attributed there to Neary and Woods.

Why the finite-table learning bound holds

Proposition 4 (Learning a finite table)

Draw n independent labeled examples from m possible inputs, each with probability at least $p_{\min} > 0$. If the fitted predictor agrees with all observed labels, then its probability of an incorrect table is at most $me^{-np_{\min}}$.

Proof. If every input appears, consistency gives the correct table. Writing p_j for the probability of input u_j , the union bound gives

$$\begin{aligned}\mathbb{P}(\text{an incorrect table}) &\leq \sum_{j=1}^m \mathbb{P}(u_j \text{ is absent}) \\ &= \sum_{j=1}^m (1 - p_j)^n \leq me^{-np_{\min}}.\end{aligned}$$

Thus $n \geq \log(m/\eta)/p_{\min}$ suffices for failure probability at most η .

The core language: syntax and conventions

Expressions have grammar

$$\begin{aligned} e ::= & o \mid \text{LookupVar}(x) \mid \text{App}(f, e_1, \dots, e_k) \\ & \mid \text{IF}(e_1, e_2, e_3) \mid \text{EQUAL}(e_1, e_2) \\ & \mid \text{HasAttr}(e, a) \mid \text{LookupAttr}(e, a) \\ & \mid \text{Assert}(e_1, a, e_2) \mid \text{SEQ}(e_1, e_2) \mid \text{Try}(e_1, e_2). \end{aligned}$$

Objects include four distinct constants `true`, `false`, `unit`, `fail`. Each procedure body has only its parameters as free variables; parameters are distinct.

We use left-to-right call-by-value evaluation. `IF` evaluates only the selected branch. A missing variable, procedure, or attribute is a stuck configuration, not the explicit value `fail`.

The semantics is deterministic on well-formed configurations. These choices make a precise lecture core; the paper does not supply a complete executable specification of all edge cases.

A continuation records one unfinished operation

Let K be a finite stack of frames, with $F :: K$ placing F on top.

$$\begin{aligned} F ::= & \text{If}(e_2, e_3) \mid \text{Seq}(e_2) \mid \text{Try}(H_0, e_2) \\ & \mid \text{EqL}(e_2) \mid \text{EqR}(o_1) \\ & \mid \text{Has}(a) \mid \text{Get}(a) \\ & \mid \text{SetL}(a, e_2) \mid \text{SetR}(o_1, a) \\ & \mid \text{Args}(f, \bar{o}, \bar{e}) \mid \text{Return}(\rho_0). \end{aligned}$$

$\bar{o}$ contains evaluated arguments; $\bar{e}$ contains the remaining ones. $\text{Return}(\rho_0)$ stores the caller's environment.

A terminal configuration is $\langle H, \rho, o, [] \rangle$. We write $\sigma \mapsto \sigma'$ for one transition and $\mapsto^*$ for zero or more transitions.

The next rule depends on the active expression or on the top frame.

Core rules: lookup, conditionals, sequencing

In these rules, the procedure table is fixed and omitted.

$$\begin{aligned}\langle H, \rho, \text{LookupVar}(x), K \rangle &\longmapsto \langle H, \rho, \rho(x), K \rangle, \\ \langle H, \rho, \text{IF}(e_1, e_2, e_3), K \rangle &\longmapsto \langle H, \rho, e_1, \text{If}(e_2, e_3) :: K \rangle, \\ \langle H, \rho, \text{true}, \text{If}(e_2, e_3) :: K \rangle &\longmapsto \langle H, \rho, e_2, K \rangle, \\ \langle H, \rho, o, \text{If}(e_2, e_3) :: K \rangle &\longmapsto \langle H, \rho, e_3, K \rangle \quad (o \neq \text{true}), \\ \langle H, \rho, \text{SEQ}(e_1, e_2), K \rangle &\longmapsto \langle H, \rho, e_1, \text{Seq}(e_2) :: K \rangle, \\ \langle H, \rho, o, \text{Seq}(e_2) :: K \rangle &\longmapsto \langle H, \rho, e_2, K \rangle.\end{aligned}$$

Each expression descends into the next required subexpression; its frame determines how the returned object is used.

Core rules: attribute reads and writes

$$\begin{aligned}\langle H, \rho, \text{LookupAttr}(e, a), K \rangle &\longmapsto \langle H, \rho, e, \text{Get}(a) :: K \rangle, \\ \langle H, \rho, o, \text{Get}(a) :: K \rangle &\longmapsto \langle H, \rho, H(o, a), K \rangle, \\ \langle H, \rho, \text{Assert}(e_1, a, e_2), K \rangle &\longmapsto \langle H, \rho, e_1, \text{SetL}(a, e_2) :: K \rangle, \\ \langle H, \rho, o_1, \text{SetL}(a, e_2) :: K \rangle &\longmapsto \langle H, \rho, e_2, \text{SetR}(o_1, a) :: K \rangle, \\ \langle H, \rho, o_2, \text{SetR}(o_1, a) :: K \rangle &\longmapsto \langle H[(o_1, a) \mapsto o_2], \rho, \text{unit}, K \rangle.\end{aligned}$$

Writes replace the value of a heap entry. A serialized list of writes implements this map by choosing the most recent matching entry.

Core rules: tests

$$\begin{aligned}\langle H, \rho, \text{HasAttr}(e, a), K \rangle &\longmapsto \langle H, \rho, e, \text{Has}(a) :: K \rangle, \\ \langle H, \rho, o, \text{Has}(a) :: K \rangle &\longmapsto \langle H, \rho, b, K \rangle,\end{aligned}$$

where $b = \text{true}$ if $(o, a) \in \text{dom}(H)$, and $b = \text{false}$ otherwise.

$$\begin{aligned}\langle H, \rho, \text{EQUAL}(e_1, e_2), K \rangle &\longmapsto \langle H, \rho, e_1, \text{EqL}(e_2) :: K \rangle, \\ \langle H, \rho, o_1, \text{EqL}(e_2) :: K \rangle &\longmapsto \langle H, \rho, e_2, \text{EqR}(o_1) :: K \rangle, \\ \langle H, \rho, o_2, \text{EqR}(o_1) :: K \rangle &\longmapsto \langle H, \rho, b, K \rangle,\end{aligned}$$

where $b = \text{true}$ if $o_1 = o_2$, and $b = \text{false}$ otherwise.

Core rules: evaluate arguments, then enter the body

For a nonempty argument list,

$$\langle H, \rho, \text{App}(f, e_1, \dots, e_k), K \rangle \longmapsto \langle H, \rho, e_1, \text{Args}(f, [], [e_2, \dots, e_k]) :: K \rangle.$$

On returning an argument, either continue evaluating arguments,

$$\langle H, \rho, o, \text{Args}(f, \bar{o}, [e, \bar{e}]) :: K \rangle \longmapsto \langle H, \rho, e, \text{Args}(f, [\bar{o}, o], \bar{e}) :: K \rangle,$$

or enter the procedure. If $\Delta(f) = ((x_1, \dots, x_k), e_f)$, then

$$\begin{aligned} &\langle H, \rho, o_k, \text{Args}(f, [o_1, \dots, o_{k-1}], []) :: K \rangle \\ &\quad \longmapsto \langle H, [x_1 \mapsto o_1, \dots, x_k \mapsto o_k], e_f, \text{Return}(\rho) :: K \rangle. \end{aligned}$$

A zero-argument call enters its body with the empty environment. A completed body restores the environment stored by Return, as on the main call slide.

The call transition in the main slides uses $\longmapsto^*$ to include these argument-collection steps.

Rollback preserves the state from before the attempted branch

$$\begin{aligned}\langle H, \rho, \text{Try}(e_1, e_2), K \rangle &\longmapsto \langle H, \rho, e_1, \text{Try}(H, e_2) :: K \rangle, \\ \langle H', \rho, \text{fail}, \text{Try}(H, e_2) :: K \rangle &\longmapsto \langle H, \rho, e_2, K \rangle, \\ \langle H', \rho, o, \text{Try}(H, e_2) :: K \rangle &\longmapsto \langle H', \rho, o, K \rangle \quad (o \neq \text{fail}).\end{aligned}$$

For example, start with $H(o, a) = \text{false}$ and evaluate

$$\text{Try}(\text{SEQ}(\text{Assert}(o, a, \text{true}), \text{fail}), \text{LookupAttr}(o, a)).$$

The result is false, and the final heap still satisfies $H(o, a) = \text{false}$.

The continuation needs both the fallback expression and the saved heap.

The bit-flipping example in the core language

Write $v = \text{LookupVar}(x)$. The body of $\text{flip}(x)$ is

```
SEQ(Assert( $v$ , bit, IF(LookupAttr( $v$ , bit), false, true)),  
    IF(HasAttr( $v$ , next), App(flip, LookupAttr( $v$ , next)), unit)).
```

Proposition 5 (List traversal)

For a nonempty finite acyclic list with Boolean bit values, this program terminates, returns unit, flips each bit once, and leaves all other heap entries unchanged.

Proof. Induct on list length. The write flips the first bit. For length one there is no next attribute. Otherwise the next object starts the shorter remaining list, and the recursive call has the claimed effect by induction.

How the language can simulate a Turing machine

Represent a sufficiently long finite tape segment by linked objects with `left`, `right`, and `symbol` attributes.

Use one procedure $f_q(x)$ for each control state q . If $\delta(q, b) = (b', +1, q')$, the corresponding branch performs

$$\text{SEQ}(\text{Assert}(x, \text{symbol}, b'), \text{App}(f_{q'}, \text{LookupAttr}(x, \text{right}))),$$

where x abbreviates its variable lookup. Use `left` for a left move; the halting branch returns `unit`.

Every finite computation can be simulated with enough supplied tape objects.

An unbounded simulation needs an unbounded supply of fresh cells, for example through allocation. The reported experiments use supplied finite object memory.

Lecture construction. Xu et al., Section 2.3, discuss preallocated heap memory and an alternative allocating pairing primitive.

More distractors require a larger retrieval margin

Suppose attention should retrieve one value, at position j_* . With scores s_j and inverse temperature $\beta > 0$, its weight is

$$a_{j_*} = \frac{e^{\beta s_{j_*}}}{\sum_{j=1}^m e^{\beta s_j}}.$$

If $s_{j_*} - s_j \geq \Delta > 0$ for all $j \neq j_*$, then

$$a_{j_*} \geq \frac{1}{1 + (m-1)e^{-\beta\Delta}}.$$

For $m \geq 2$ and $0 < \varepsilon < 1$, $a_{j_*} \geq 1 - \varepsilon$ is guaranteed when

$$\beta\Delta \geq \log \frac{(m-1)(1-\varepsilon)}{\varepsilon}.$$

Increasing the context can change the accuracy of a familiar retrieval rule.

What would a more informative length experiment measure?

Hold the interpreter and decoding protocol fixed. Vary separately:

1. **Duration:** more iterations with similar live state size.
2. **Memory:** more objects and bindings at similar execution depth.
3. **Retrieval:** more distractors and greater distance to the relevant entry.
4. **Composition:** new nesting, variable shadowing, and write/read patterns.

Measure exact completion accuracy and exact full-execution accuracy, alongside token accuracy, and record the first error.

These tests identify which shared rule stops transferring, and under what change.

Can execution introduce a new procedure name?

The required state operation is simple:

$$\Delta_{t+1} = \Delta_t[f \mapsto ((x_1, \dots, x_k), e)].$$

A later $\text{App}(f, o_1, \dots, o_k)$ retrieves this body and uses the same call rule as before.

The new body belongs in writable program memory.

In the reported MicroPy setup, procedure definitions are supplied in advance; writes update object attributes.

A natural extension is to construct a body as data, bind it to a name, and execute later calls. This adds a new learning question: can the model preserve and use definitions created during the computation?

A sign discrepancy in the printed Schuurmans program

For state G reading 0, compare the specification with the printed code:

Table 1	$\delta(G, 0) = (0, +1, H)$	move right,
prompt G	$(0, -1, H)$	move left,
displayed response, test 12	$(0, -1, H)$	move left.

The displayed model response follows the prompt, but that prompt disagrees with the stated Turing machine.

The simulation theorem requires agreement with the transition table.

Our construction uses Table 1. Matching the printed implementation to that table requires correcting this sign and verifying the revised prompt.

Schuurmans (2023): Table 1, Section 4's definition of G , and Section 5's verification test 12.

Reading the empirical result precisely

Test inputs. The four bit tasks use lengths 2–10. SAT solving uses 2–4 variables and 1–6 clauses; SAT verification has 36 instances.

Metric. Tables 1–2 define accuracy as token-level success and report 100% for each task. They do not provide raw token-error counts or a detailed decoding protocol. Exact-prefix induction states the consequence of literal zero errors under a fixed decoder and the same context protocol.

Length. The 128-line cap is stated for generator-sampled training executions. Targeted plan sampling additionally supplies difficult local configurations. No training-versus-test context-length distribution is given.

Memory. Live state and effects are serialized; PENCIL performs syntactic contractions. The memory benefit depends on retaining a sufficient, compact returned payload, including changes to mutable state.

Xu et al., Sections 2–3, Tables 1–3, and Appendix B. These source details are useful for interpreting and extending the experiment.